

CHAPTER 16

Simulation

16.1. Introduction

The operations research tools introduced in Chapters 14 and 15 are designed for use with problems for which tractable, closed-form solutions may be found. These are typically called *analytical solutions*. In engineering practice, many situations arise involving problems for which no analytical solution can be obtained. Analytical solutions have been derived successfully when dealing with narrowly defined problems residing in relatively small subsystems or components of the total organization. Many problems must be viewed in a larger context, leading to a degree of complexity beyond the ability of analytical solutions to handle.

The limitations of analytical solutions were one of the factors leading to the development of simulation as a means of dealing with complex problem situations. Another motivation for the development of simulation was the desire to be able to examine the details of the dynamics of a complex operating system. In particular, we can often gain valuable insight into system behavior by the detailed tracing of entities moving through the system.

System simulation has become the most widely used tool among industrial and systems engineers.

16.2. Simulation Examples

As an introductory example of simulation, consider the familiar experience of tossing a fair coin. The possible outcomes are "heads and tails," and each is equally likely to occur on a particular toss. Furthermore, the outcome of one toss will in no way affect the outcome of the next toss. Thus successive outcomes are considered *independent*.

Suppose that for some reason we are unable to obtain a fair coin. We do, however, have access to a list of digits (see Table 16.1) 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 that are in completely random order. We can construct a *simulation experiment* that imitates the tossing of a fair coin by assigning exactly half of the digits to correspond to a "head," and the other half to correspond to a "tail."

The probability distribution for the tossing of a fair coin is shown in Figure 16.1a. The cumulative distribution function is shown in Figure 16.1b.

If we assign the digits 0, 1, 2, 3, 4 to correspond to the outcome "head" in our simulated coin-tossing experiment, and the digits 5, 6, 7, 8, 9 to correspond to the outcome "tail," we can generate as many trials as we wish by drawing random digits from our list of random digits in Table 16.1. Digits can be drawn from this table in a variety of ways. For example, we could use the extreme-left single-digit column. The first 10 digits are 3, 8, 0, 3, 0, 9, 9, 1, 2, 2, which would correspond to H, T, H, H, H, T, T, H, H, H. In this sample, we obtain 7 heads and 3 tails: not a very close approximation of the theoretical probabilities for the two outcomes, but an entirely plausible result if we were to toss an actual coin. The problem is one of sample size. The reader should continue down the column and verify the following results:

$p(x)$	Number of Samples				
	10	20	30	40	50
H	0.7	0.6	0.47	0.45	0.50
T	0.3	0.4	0.53	0.55	0.50

Now, consider the case of tossing an unfair coin. A very large number of experiments has established that this particular coin has a probability of 0.47 of coming up "head," 0.52 of coming up "tail," and 0.01 of landing on its edge. We clearly cannot simulate these probabilities using single-digit random numbers. We must, instead, use two-digit random numbers.

The probability distribution and cumulative distribution functions for this case are shown in Figure 16.2a and b. To perform our simulation experiments, we will use the extreme-left single-digit column and the digit in the adjacent column to form our two-digit random numbers. The first two are 36 and 89. The digits 00–46 will represent a "head," digits 47–98 will represent a "tail," and digit 99 will represent an "edge." The reader should verify the following outcomes:

Table 16.1. HAND-DRAWN RANDOM DIGITS

36550	98056	23736	94727	34879	47261	12538	25757	34258	50167	95802	50562	24401
89796	07620	47787	86649	70937	64585	36855	40407	28721	07273	23007	49725	37646
05983	50001	91826	82914	03235	41536	72104	90021	04880	15607	37422	28986	68156
34415	34393	86438	89537	17041	96369	40309	19173	74179	70516	25286	34218	68319
05648	32013	46902	78910	96560	86010	90506	89154	99909	06901	43675	30794	40072
90740	24858	38917	62201	25396	42331	10262	03001	94234	48033	05899	83159	08628
94697	10437	85219	76188	05292	30189	58429	77799	98158	39084	92137	08623	35271
13199	80107	94071	73054	14619	05779	05967	37654	04510	11663	35196	50849	39540
29622	94207	18958	12650	05160	36521	77625	89042	12010	46724	54309	85235	58677
22504	92303	50156	62504	51028	79390	64878	49920	14898	05145	99219	07882	71913
18354	32762	90334	97817	67887	83203	66395	54369	01498	02864	50063	93443	28268
61978	79644	20051	32057	23904	99112	94519	35138	18440	73147	83445	01340	53947
83773	69718	71520	61986	92535	70426	32567	33471	77371	58059	81488	66028	44411
94864	74318	77919	13728	73108	24038	56958	01136	36537	88984	81581	13442	78662
05174	83531	15425	98442	88428	68645	06533	41574	37378	75647	70959	73571	55543
51602	63904	13242	80612	37318	67352	74609	29663	17106	58806	47142	32473	36843
87636	04993	61505	65126	06915	13441	84627	17411	66290	32692	70359	19974	18478
27932	36660	60331	18861	06090	14290	61498	38418	92613	80466	22598	32021	62896
37237	30304	90098	15925	52730	59982	85657	29982	76637	91637	59043	00661	47037
22683	26320	28745	14753	09337	92193	72088	25069	61236	21574	67589	25827	30987
58178	66217	25095	26812	65662	96589	13613	81059	58664	18280	46826	09001	44014
60628	38526	81246	73004	58480	44765	30804	49710	60768	61120	80775	56051	76291
71783	17403	24911	89961	37526	29216	91847	67142	98210	42045	81985	29970	63734
40720	24312	56871	13170	56642	19816	23803	84668	77796	19555	54025	44325	77635
67671	30334	02293	18773	90515	44496	14903	53865	70311	26413	37990	17265	24121
58878	96540	23409	49122	39104	76370	28587	27108	64678	45937	34707	03351	34034
81323	58722	28368	84258	02462	13631	37409	06978	75540	53513	90336	46498	82952
56005	62356	32617	94365	30817	90307	52348	61557	52833	19295	73552	62830	01247
01045	15189	66867	32947	33578	82258	15844	93901	13277	09747	01625	78190	25780
73551	57253	49467	73742	70703	13393	16199	84230	69048	93226	10983	65077	18264
59850	15350	15526	46134	37527	29436	61470	76626	71550	43323	52173	83869	50438
61217	51080	15270	97588	81424	55470	95942	60926	33425	05774	77328	27584	21160
06249	30048	70208	36398	28538	71455	98580	64586	32889	35665	55124	64594	56457
28627	88560	52007	24839	24453	15843	99620	14738	56309	67421	91906	81140	62461
93447	22741	93380	05956	14780	87760	50253	41125	88069	17036	37914	83832	75007
12669	91704	65678	93611	35240	28095	97447	24843	35980	38654	83066	82453	66846
96252	90334	85665	03746	07454	95413	67678	34295	96265	28131	10493	90775	19611
86346	32782	44391	24731	84592	50880	99935	97335	72085	11823	47547	81435	85442
10147	32169	30187	35742	94051	11313	53564	88306	56333	41244	90012	92130	07836
87898	82498	07194	04596	63575	74322	57189	16562	85534	68843	37290	33940	65369
47158	93692	53645	68529	56366	77735	20208	14514	05314	36795	49869	05092	94981
46915	53514	16696	60012	25648	69902	01822	81820	38032	36275	67910	72884	26626
66512	58580	91015	87784	09244	34806	12695	57722	19418	88528	25450	29532	63215
45444	15844	31068	76729	24516	31870	53130	32184	29121	38728	91528	88755	44495
71801	95118	98230	26211	72970	82872	13659	55413	57741	87062	72094	03092	36030
58688	03020	24492	64330	33429	76947	04468	79148	62968	54914	69992	63234	42738
17282	10399	28449	01839	32812	81561	25847	82339	14984	56133	22374	99746	90023
14817	84395	79463	00923	95064	36755	75044	40233	26347	46475	15462	53778	81041
37253	27111	87904	90105	24805	39220	86661	11434	03596	43846	79784	74490	09823
06625	08048	05250	31351	67023	09525	31724	85009	85535	20758	23600	77615	70627

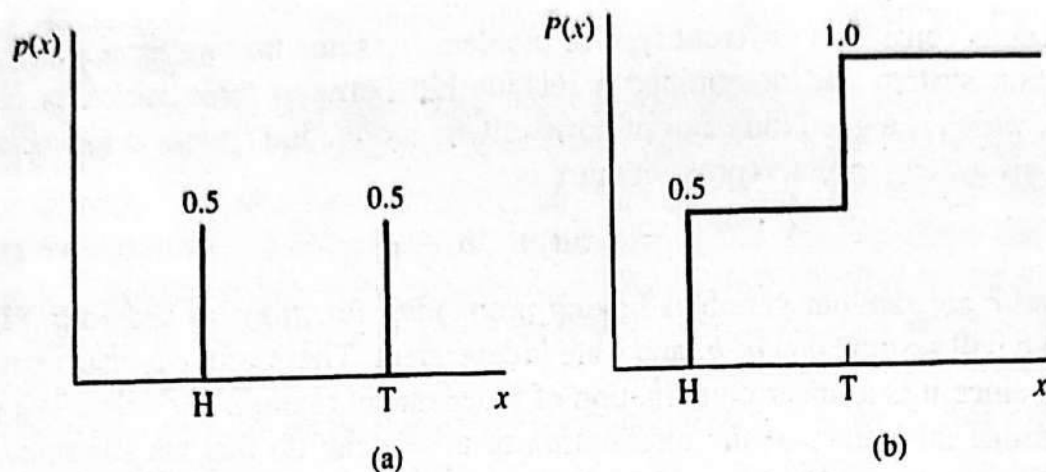


Fig. 16.1. Probability distribution and cumulative distribution functions for tossing of a fair coin.

$p(x)$	Number of Samples				
	10	20	30	40	50
H	0.7	0.6	0.47	0.45	0.48
T	0.3	0.4	0.53	0.55	0.52
E	0	0	0	0	0

The results are only slightly different from before, because the probabilities are only slightly different. We did not encounter an outcome of "edge" in our 50 sample experiment, but if we continued the experiment for a very long time, we would expect 1% of the outcomes to be "edge."

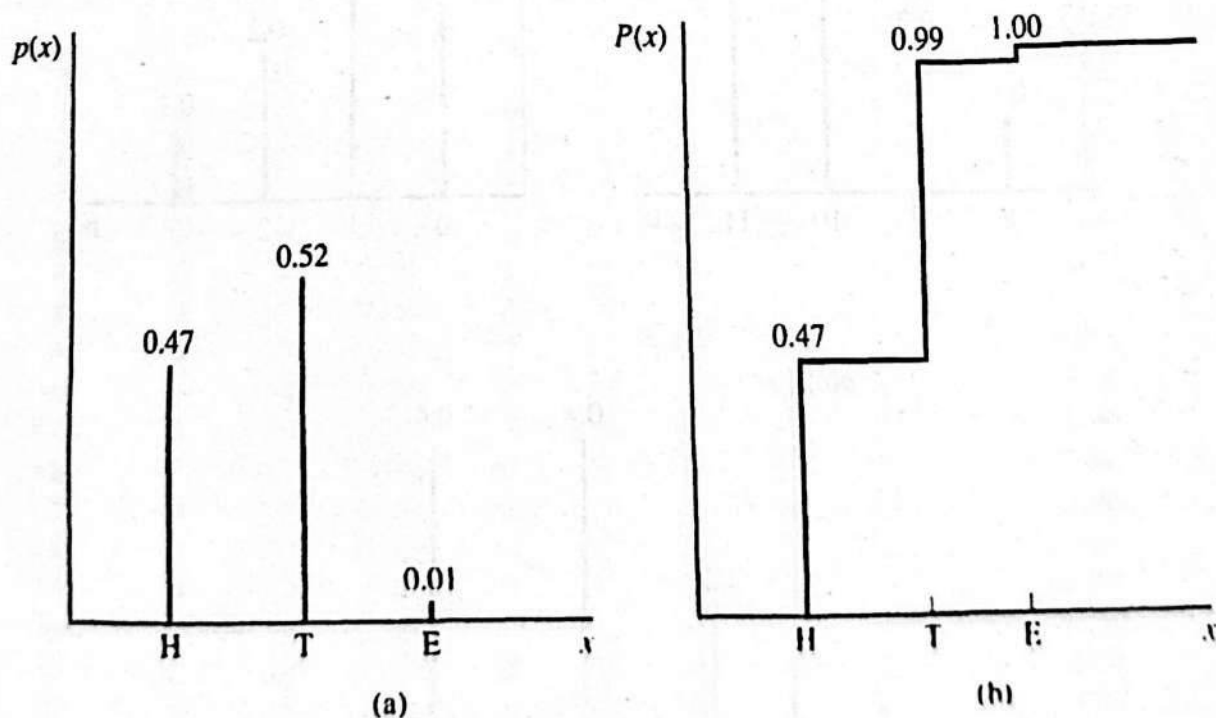


Fig. 16.2. Probability distribution and cumulative distribution functions for tossing an unfair coin that may land on edge.

Let us consider a different type of problem. Assume that we have analyzed a production system and determined a relationship between three variables in the system, which is a good indicator of how well the production system is functioning.

This system effectiveness measure is

$$e = 1.5a + 3b + c \quad (16.1)$$

a , b , and c are random variables having probability functions as shown in Figure 16.3. We will assume that a , b , and c are independent. Therefore, e is also a random variable since it is a linear combination of independent random variables. We wish to determine the nature of the distribution of e . We can do this via simulation by conducting an experiment in which a number of observations of a , b , and c are generated using the general procedure employed in the previous example. These sets of values will be substituted into Equation (16.1), giving a number of values of e . We can then analyze these values of e , just as we would any set of statistical data collected from the "real world." We can plot the frequency distribution, calculate the mean and standard deviation, and determine whether the distribution of e is of some standard form.

Using the same scheme employed in the previous example, we will assign digit groupings to correspond to the probabilities of the possible outcomes of the random variables. For variable a , digit 0 will correspond to "8," digits 1–2 to

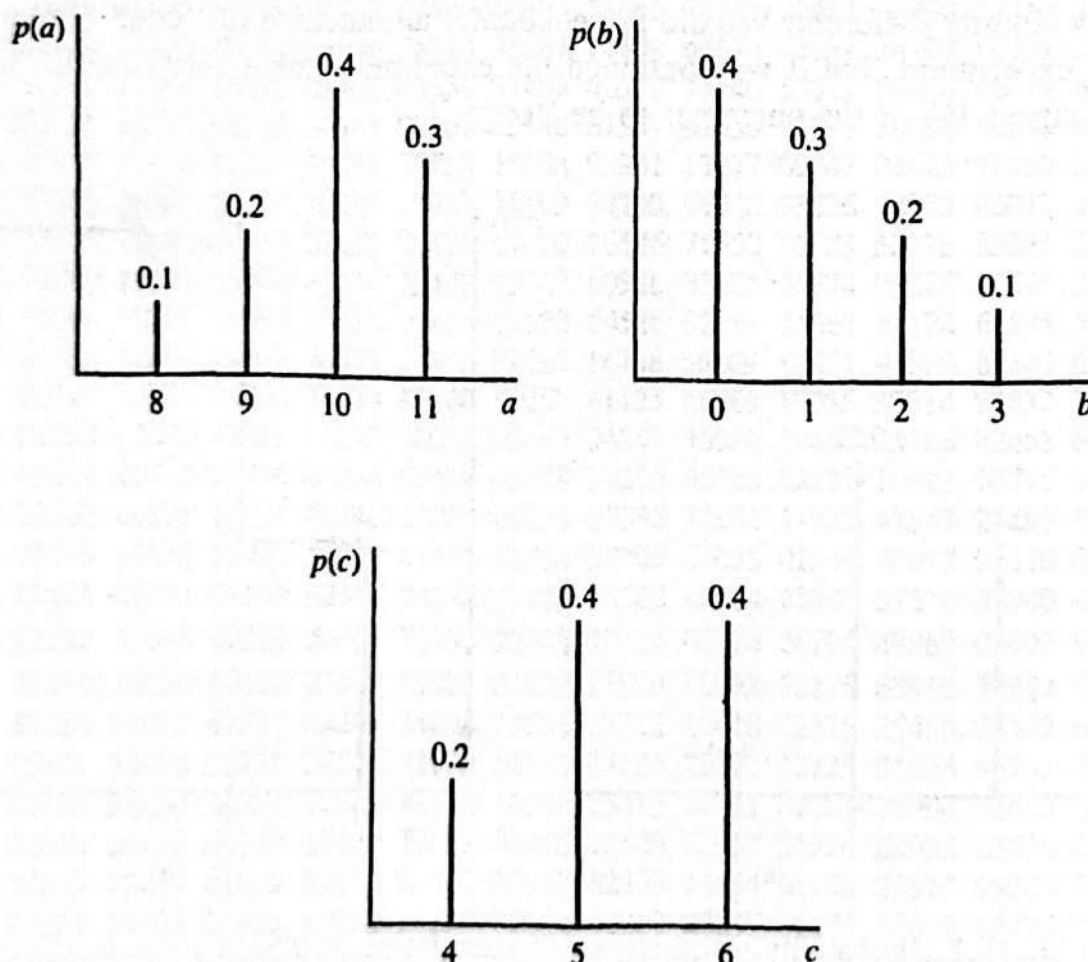


Fig. 16.3. Probability distributions for variables a , b , and c .

"9," digits 3–6 to "10," and digits 7–9 to "11." Similar assignments are made for variables b and c .

Since we are now sampling from three probability distributions, we should use a different "stream" of random digits from Table 16.1. This can be done in a number of ways. For this case, we will select three columns of five-digit numbers and use the rightmost digit from each group. Let us select the first three columns. Column 1 is used to sample from variable a , column 2 from variable b , and column 3 from variable c . The first set of digits are [0,6,6], the second set are [6,0,7], the third set are [3,1,6], and so on.

The results of a 20-sample experiment are shown in Table 16.2. The resulting probability distribution of e is shown in Figure 16.4. Obviously, a much larger sample size would be required to obtain a realistic probability distribution.

As a more realistic example, suppose that we have a set of machines and, with equal likelihood, either none or one breaks down per hour. Half of the breakdowns require $1\frac{1}{2}$ hours to repair, and the other half require $2\frac{1}{4}$ hours to repair. For each minute a machine is out of service the company loses \$1. Repairpersons for these machines are each paid \$6 per hour, whether or not the repairpeople are working. Only one repairperson may work on a given machine at a time. The objective of our experiment is to determine the number of repairpeople we should employ in order to minimize total costs.

We shall simulate the performance of this machine–repairperson system for a period of 10 hours. First, we shall perform the simulation with one repairperson

Table 16.2. SIMULATION EXPERIMENT FOR EQUATION (16.1)

Sample Number	a			b			c		$e = 1.5a + 3b + c$
	R.N.	a	$1.5a$	R.N.	b	$3b$	R.N.	c	
1	0	8	12.00	6	1	3	6	6	21.00
2	6	10	15.00	0	0	0	7	6	21.00
3	3	10	15.00	1	0	0	6	6	21.00
4	5	10	15.00	3	0	0	8	6	21.00
5	8	11	16.50	3	0	0	2	5	21.50
6	0	8	12.00	8	2	6	7	6	24.00
7	7	11	16.50	7	2	6	9	6	28.50
8	9	11	16.50	7	2	6	1	4	26.50
9	2	9	13.50	7	2	6	8	6	25.50
10	4	10	15.00	3	0	0	6	6	21.00
11	4	10	15.00	2	0	0	4	5	20.00
12	8	11	16.50	4	1	3	1	4	23.50
13	3	10	15.00	8	2	6	0	4	25.00
14	4	10	15.00	8	2	6	9	6	27.00
15	4	10	15.00	1	0	0	5	5	20.00
16	2	9	13.50	4	1	3	2	5	21.50
17	6	10	15.00	3	0	0	5	5	20.00
18	2	9	13.50	0	0	0	1	4	17.50
19	7	11	16.50	4	1	3	8	6	25.50
20	3	10	15.00	0	0	0	5	5	20.00

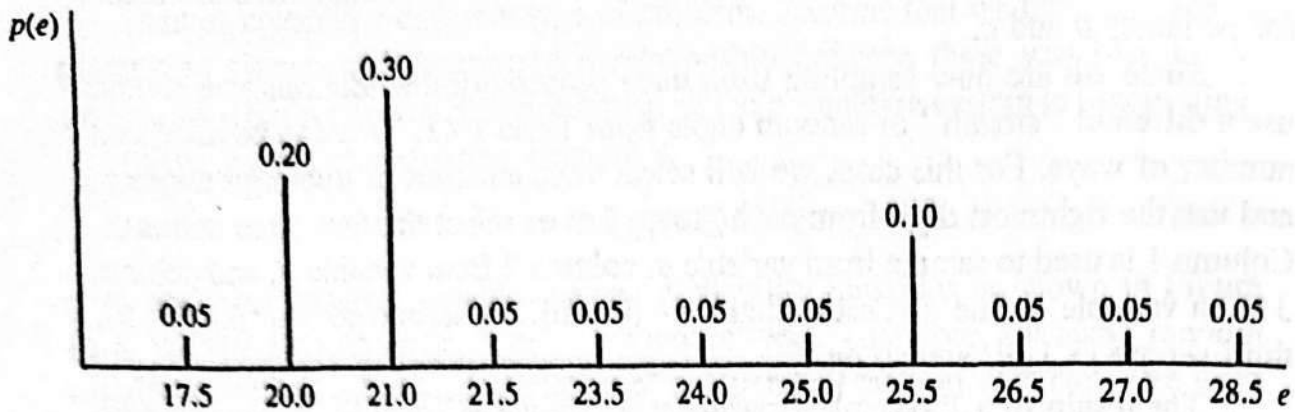


Fig. 16.4. Observed probability distributions of e from Equation (16.1).

and measure the resulting total cost; then we shall use two repairpeople and measure the resulting total cost; and so on. The equation for calculating total cost for various numbers of repairpeople, based on a 10-hour simulation, is

$$TC_k = (\$6)(10)(k) + (\$1)(DT) \quad (16.2)$$

where TC = total cost

k = number of repairpeople employed

DT = number of minutes of machine downtime

We shall obtain a value of TC for several different numbers of repairpeople and select the policy that results in lowest TC .

The simulation procedure is simple. We shall flip coin 1 and if it comes up tails, we shall assume that no machines break down. If it comes up heads, we shall assume that one machine breaks down at the beginning of the hour. If a machine breaks down, we shall flip coin 2 to determine whether the repair time is 1.5 hours (tail) or 2.25 hours (heads). Notice that one repairperson should be able to handle the job since, on the average, only one machine is breaking down each 2 hours and the average repair time is only 1.875 hours. With only one repairperson, however, we are taking a chance that several machines will break down in a short period of time or that several will require a long repair time, causing avoidable machine downtime and increasing the cost.

Table 16.3 is a tabular model of the system with which we are experimenting. We see that with one repairperson there are 11.25 hours of machine downtime (9.75 hours repair time plus 1.5 hours avoidable downtime), bringing the total cost to \$60 for the employee plus \$675 for machine downtime, for a total cost of \$735. With two repairpeople there is no avoidable downtime in our simulated example; therefore, the cost is \$120 plus \$585 or \$705. Although three repairpeople were not considered, their cost would have been \$765. The results indicated that we should hire two repairpeople, even though they will not be kept busy all the time (actually, not even half the time). Of course, these conclusions are based on a short time period, and it is customary to simulate a much longer period in order to gain confidence in the results.

Table 16.3. SIMULATED MACHINE REPAIR SYSTEM

Hour of Operation	Coin 1	Number of Breakdowns	Coin 2	Machine Repair Time (Hours)	Start Repair		End Repair		Avoidable Machine Downtime (Hours)	
					1 Man	2 Men	1 Man	2 Men	1 Man	2 Men
1	T	0								
2	H	1	T	1.5	2	2	3.5	3.5	0	0
3	H	1	H	2.25	3.5	3	5.75	5.25	0.5	0
4	T	0								
5	H	1	T	1.5	5.75	5	7.25	6.5	0.75	0
6	T	0								
7	T	0								
8	H	1	H	2.25	8	8	10.25	10.25	0	0
9	T	0								
10	H	1	H	<u>2.25</u> 9.75	10.25	10	12.5	12.25	<u>0.25</u> 1.5	<u>0</u> 0

16.3. Random Number Generation

Several assumptions were made in the previous example to vastly simplify the simulation so that it could be done by hand. For example, there were only two equally likely choices (0 and 1) of breakdowns per hour. Also, breakdowns occurred only at the beginning of the hour and repair times included only two equally likely choices (1.5 and 2.25 hours). In both cases, the value of the random variable (0 or 1; 1.5 or 2.25) was determined by a coin flip. Simulations normally involve much more complex models, and the process requires a computer.

During a simulation we frequently require the computer program to select a value of a random variable from its underlying distribution. For example, suppose that during a simulated experiment we require several "typical" values of a Poisson distributed random variable X with mean 1.5. From Appendix B we can see that the random variable will take on values of 0 with probability 0.223, 1 with probability 0.335, 2 with probability 0.251, and so on.

If we use the computer's ability to generate many three-digit *random numbers* ranging from 001, 002, up to 999 and 000, each having the same probability of occurrence, these can be used to randomly select values for our Poisson random variable X .

Consider the cumulative Poisson distribution shown in Figure 16.5. Suppose that the first three-digit number we generate is between 001 and 223, inclusive. This will "map" onto a value of $X = 0$. If the number generated ranges from 810 to 934, it will "map" onto a value of $X = 3$. For example, the number 876 is shown in Figure 16.5 to call for the value $X = 3$.

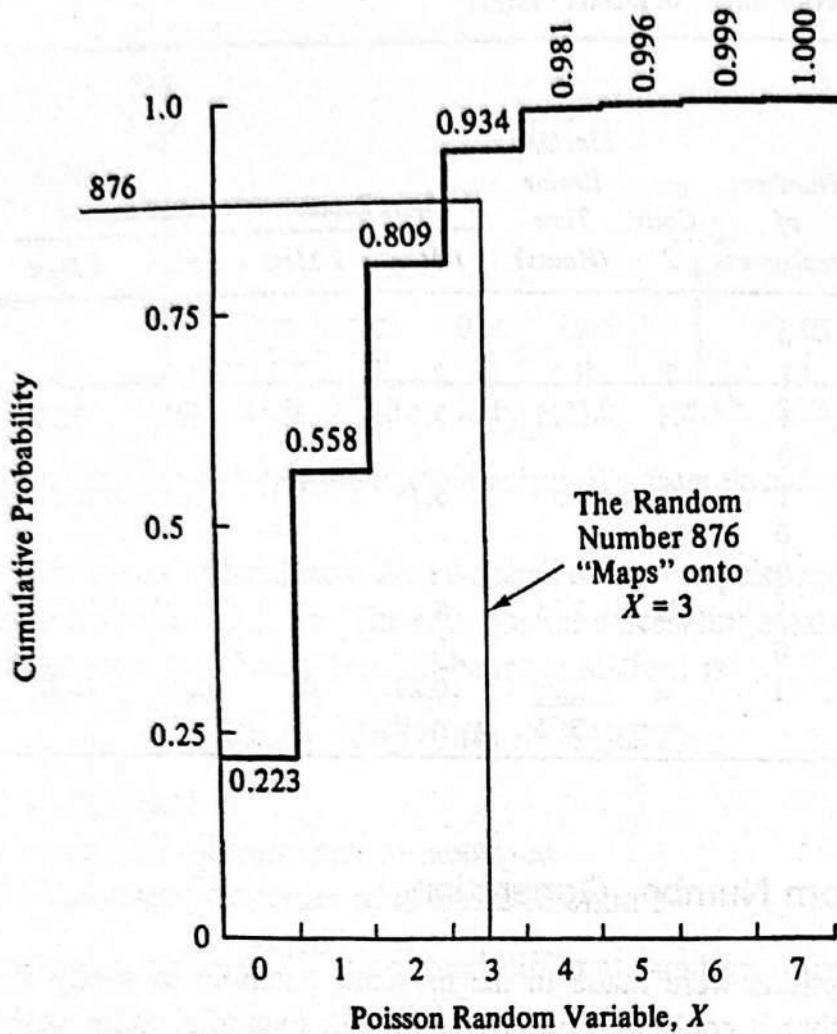


Fig. 16.5. Cumulative distribution function of Poisson distribution (mean = 1.5).

Thus far, we have considered only discrete probability distributions. Many random variables are inherently continuous. Sampling from continuous distributions can also be simulated.

Consider the exponential random variable. Its density function, $f(x)$, and cumulative distribution function, $F(x)$, are shown in Figure 16.6. We can obtain simulated sample values from the exponential by generating a random value from a uniform distribution having range $[0,1]$. This value can be transposed to the vertical axis of the cumulative distribution function, projected horizontally until it intersects the $F(x)$ curve, and then projected vertically downward to the x axis. The point of intersection is a randomly selected value of the random variable (see Figure 16.7). Note that this process is essentially identical to the one we employed for sampling from the discrete Poisson random variable in Figure 16.5.

In order to sample from continuous distributions in a computer simulation model, we need to be able to perform the operations just illustrated computationally. In other words, we need to develop an equation for generating random values that can be included in our computer program. This turns out to be very easy to do for the exponential random variable.

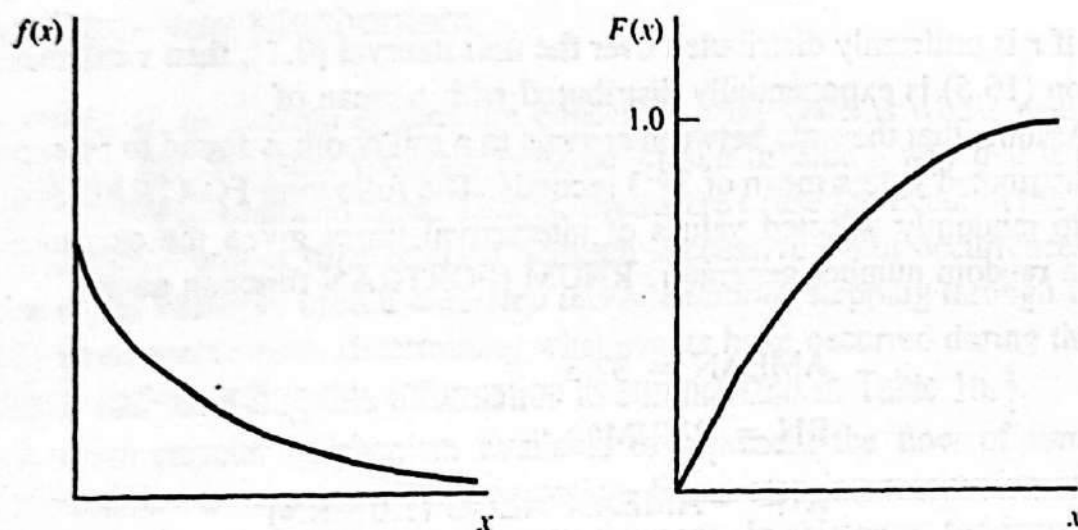


Fig. 16.6. Density function and cumulative distribution function for an exponential random variable.

The exponential random variable has a single parameter, its mean, μ . The cumulative distribution function is

$$F(x) = 1 - e^{-x/\mu} \quad (16.3)$$

An examination of Figure 16.7 reveals that $r = F(r)$ and that $F(r) = F(x)$ (both being defined on the unit interval). This allows us to form the relation

$$r = 1 - e^{-x/\mu} \quad (16.4)$$

Solving this equation for x gives

$$x = -\mu \ln(1 - r) \quad (16.5)$$

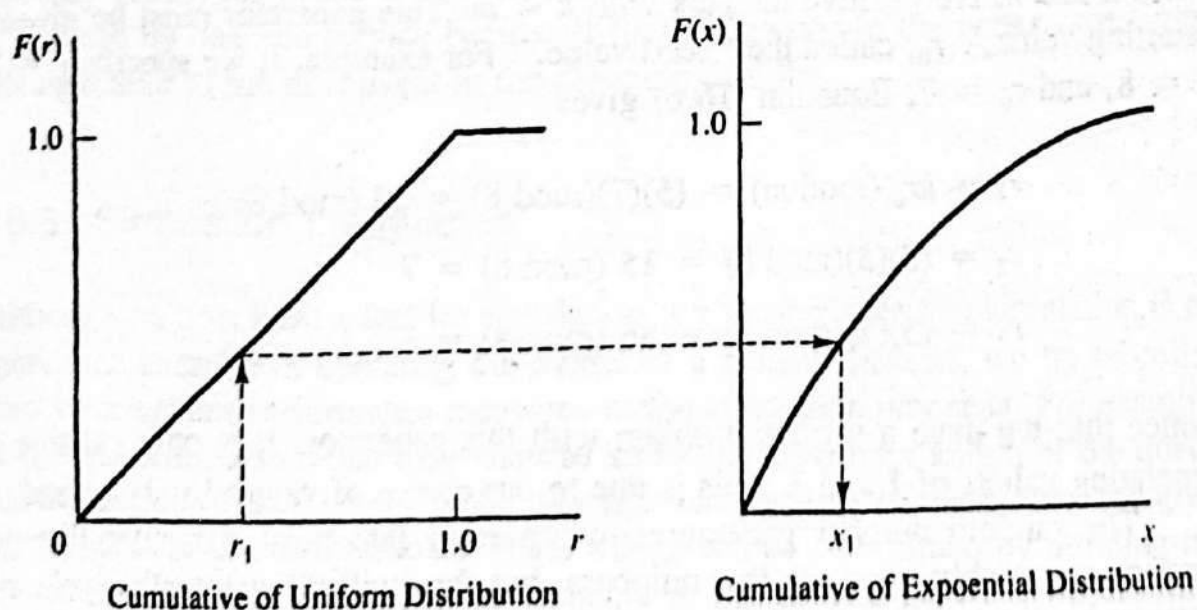


Fig. 16.7. Illustration of simulated sampling from a continuous distribution.

Hence if r is uniformly distributed over the unit interval $[0,1]$, then x calculated by Equation (16.5) is exponentially distributed with a mean of μ .

Assume that the time between arrivals to a toll booth is found to be exponentially distributed with a mean of 37.3 seconds. The following FORTRAN code will generate randomly selected values of interarrival times given the existence of a uniform random number generator, RNUM (FORTRAN function name):

```
AMEAN = 37.3
RN = RNUM( )
X = -AMEAN*ALOG (1.0 - RN)
```

The exponential function is a very easy function to work with. Others, such as the density function for the normal random variable, are more difficult, and cannot be derived in closed form. Approximations have been developed for the normal and other intractable density functions. Essentially all simulation languages include routines for automatically generating random values from a wide variety of statistical distributions, both discrete and continuous.

All such process generators depend on a random number generator that generates random values on the unit interval $[0,1]$. Many approaches have been attempted to develop good random number generators. It would be inconvenient to be restricted to using tables of previously determined random numbers, such as those in Table 16.1. We need a way to calculate a random number whenever we need one.

The most widely used random number generators are those that employ the multiplicative congruential method

$$r_{n+1} = k r_n \pmod{m} \quad (16.6)$$

where k and m are positive integers with $k < m$. This generator must be given a "starting value," r_0 , called the "seed value." For example, if we specify $k = 5$, $m = 8$, and $r_0 = 7$, Equation (16.6) gives

$$r_1 = k r_0 \pmod{m} = (5)(7) \pmod{8} = 35 \pmod{8} = 3$$

$$r_2 = (5)(3) \pmod{8} = 15 \pmod{8} = 7$$

$$r_3 = (5)(7) \pmod{8} = 35 \pmod{8} = 3$$

Notice that we have a serious problem with this generator. It is only capable of generating values of 7 and 3. This is due to our choice of values for k , m , and r_0 .

The random number generators incorporated into most computer libraries produce reasonably good random numbers, but they will all eventually cycle and start producing the same sequence again. For most simulation studies, this is not a serious problem.

16.4. Time-Flow Mechanism

Since almost all simulation studies are concerned with systems whose events are time-ordered, the computer program must be written in such a way that it moves the model through simulated time. That is, events are caused to occur in the proper order and with a proper time interval between successive event occurrences. Our machine repair example used a *time-step incrementation*, stepping through time in equal (1-hour) increments, determining what events have occurred during the time increment, and recording this information as summarized in Table 16.3.

A more popular mechanism available to represent the flow of time in a simulation model is *event-step incrementation*. Event-step incrementation calls for a simulation to proceed from one event to the next. In this case, the incremental time steps are generally uneven. The simulation begins at time zero, and the occurrence times of the events resulting from the simulated performance of all system components are determined. The *master clock* is updated to the time of the earliest event occurrence. Simulated time advances to this point, the new system state is determined, and subsequent event occurrence times are determined for the processes for which events have just occurred. The simulation then advances to the time of the next earliest event occurrence, and the process repeats itself throughout the simulation.

An example of event-step incrementation is the flow chart of a queueing system having one service facility and infinite queue capacity as illustrated in Figure 16.8. When a customer arrives for service, before being assigned to a server, we first generate the time of our next arrival and enter that arrival chronologically in the *event list*. Then, if the server is free, we assign our customer to the server, show the server as busy, and generate the length of the service. We then enter the end-of-service time in the event list. If the server is busy, we increase the queue size by one and our customer must wait. When a service ends, the server is removed from the busy status, the queue is checked, and if another customer is waiting, the line is reduced by one and service begins. If there is no one in the queue, we simply advance time to the next event in the chronological list.

16.5. Simulation Languages

Although we now have a feel for simulation, we must realize that simulation is not merely an exercise in operating the model of a system. Instead, we try to collect important system performance measures as the simulation proceeds. For example, in the queueing system we may want to know the maximum length of the queue, the utilization of the server, or the average waiting time in the system for given arrival and service distributions. These things can be determined by building into the computer program appropriate counters or registers to gather these statistical data. Better yet, we can make use of available simulation languages to do this work

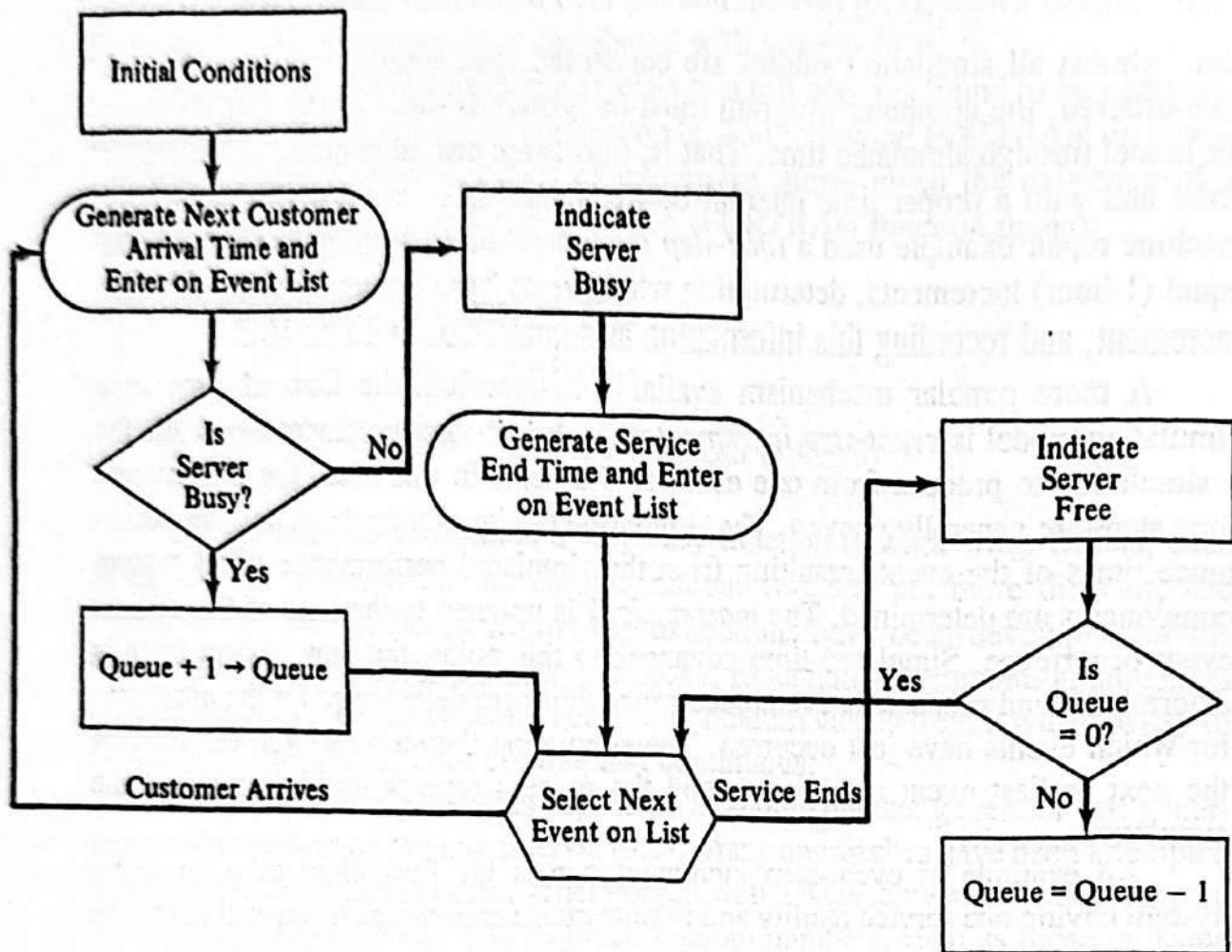


Fig. 16.8. Variable increment simulation flowchart.

for us. Simulation languages provide specific concepts and procedures for describing a system *and* moving it from state to state over time. Three relatively well-known simulation languages are GPSS, SLAM, and SIMSCRIPT.¹

GPSS (General-Purpose Simulation System) has broad appeal due to its modeling simplicity. A GPSS model is developed by forming a block diagram from over 40 standard operational blocks, such as “generate,” “terminate,” and “advance.” GPSS programming consists primarily of learning the functions of the GPSS blocks and how to combine them logically to represent the desired system.

SLAM (Simulation Language for Alternative Modeling) is an advanced FORTRAN-based language that represents the state of the art in simulation languages of interest to the I.E. at the time. It provides network symbols for building graphical models, and it contains subprograms for discrete event (as presented in this chapter) and continuous modeling developments. Extremely complex problems of interest to practicing industrial engineers can be well handled using SLAM.

SIMSCRIPT II is a simulation language divided into five levels:

¹A review of these and other simulation languages is presented in A. Alan, B. Pritsker, *Introduction to Simulation and SLAM*, 3rd ed. (New York: Halsted Press, 1986).

- (1) A simple teaching language to introduce programming concepts.
- (2) Statement types of power comparable to FORTRAN.
- (3) Statement types of power comparable to ALGOL or PL/I.
- (4) Statement types that provide a structure for modeling.
- (5) Statement types for advancing time, processing events, generating samples, and accumulating data statistics.

SIMSCRIPT programs are easy to read and tend to be self-documenting, due to their English-like form.